

Circuit Simulation Journey: A Tale of Tools, Ethics

By: Hassani Fateh & Kahlrras Raid
For: Mr Sahraoui Walid & Mr BOUCHACI Islem

December 2025

1 Introduction: Why We're Here

Before building any circuit in real life, we needed to simulate everything. Simulation helps us:

- Understand how the circuit behaves
- Check voltages and signals
- Find mistakes early
- Make sure the design works as expected

Simple, right? **Wrong!**

This report is the story of how we tried to simulate three analog/digital circuits and ended up learning more about software tools, ethics, and teamwork than we ever expected.

Spoiler: It took us through 4 different tools before we finally got it working.

2 The Circuits We Needed to Simulate

Before we talk about the chaos, let's explain what we were actually trying to build. We had three circuits to simulate, each with its own challenges.

2.1 Circuit 1: Switched-Capacitor / Transmission-Gate Sampling Network

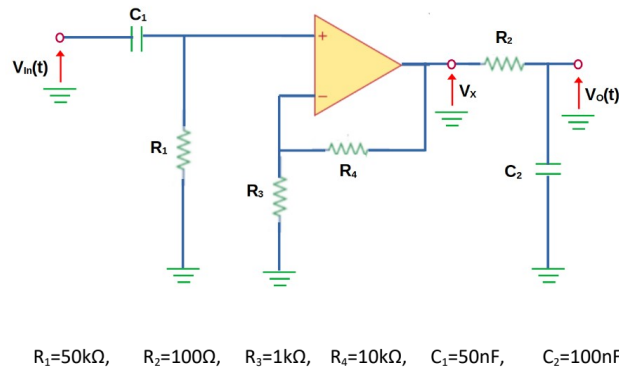


Figure 1: Circuit 1: Switched-Capacitor Sampling Network

2.1.1 What it is

A tiny sample-and-hold / charge-transfer block. We connect the input to a capacitor with a transmission gate (analog switch) to sample the voltage, then we isolate the cap and connect its other plate to the output so charge moves and the output updates. There's a resistor to ground that slowly bleeds the held voltage.

2.1.2 What happens during simulation (step by step)

1. **Sample phase (ϕ_1 closed):** Input V_1 is tied to the capacitor left plate. The cap charges to approximately V_1 .
 - *Waveform:* The left-plate voltage follows V_1 while switch closed.
2. **Switching ($\phi_1 \rightarrow \phi_2$):** Left switch opens, right switch closes to transfer charge to the output bus. Charge redistributes between the sampling cap and output capacitance $\rightarrow$ output jumps to new value.
 - *Waveform:* Output node shows a step (stair) or jump when transfer happens.
3. **Hold / droop:** With output node connected to resistor R to ground, output decays exponentially: $V_4(t) = V_4(0) \cdot e^{-t/(RC_{eq})}$
 - *Waveform:* Slow exponential decay (droop).
4. **Repeated sampling:** Next ϕ_1 recharges the cap and repeats.

2.1.3 Shapes & events to probe

- Probe input V_1 : whatever test signal we apply (DC step, sine, whatever)
- Probe left-plate: track charging during ϕ_1 , a quick RC charge curve if switch has R_{on}

- Probe output V_4 : step when transfer happens; then exponential decay until next sample
- If you zoom around the switching instant you may see tiny spikes from charge injection or clock feedthrough

2.1.4 Useful formulas

- Charge sharing final voltage: $V_{out} = \frac{C}{C+C_{out}} \cdot V_{sample}$
- Droop time constant: $\tau = R \cdot C_{eq}$
- Example numbers: If $C = 10$ nF and $R = 1$ M Ω , then $\tau = 10$ ms so droop is obvious on the ms scale

2.1.5 Common sim problems we expected

- No ground $\rightarrow$ simulator errors
- Switch model missing $\rightarrow$ need proper analog switch or MOS transmission gate models
- Clock timing wrong $\rightarrow$ ensure ϕ_1 and ϕ_2 are non-overlapping
- Charge injection visible $\rightarrow$ expected behavior; reduce by using transmission gate (NMOS+PMOS)

2.2 Circuit 2: AC-Coupled Non-Inverting Op-Amp + Output RC Filter

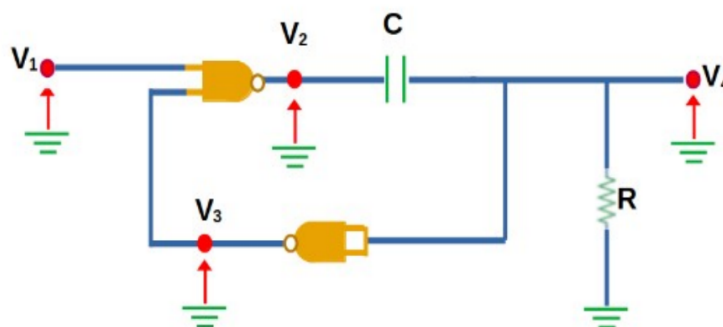


Figure 2: Circuit 2: AC-Coupled Op-Amp with Output Filter

2.2.1 What it is

A non-inverting amplifier that blocks DC on the input (C_1) and has a small low-pass at the output ($R_2 + C_2$). Gain is set by R_3 and R_4 .

Given values:

- $R_1 = 50$ k Ω , $C_1 = 50$ nF $\rightarrow$ input high-pass
- $R_3 = 1$ k Ω , $R_4 = 10$ k $\Omega \rightarrow$ gain $= 1 + R_4/R_3 = 11$
- $R_2 = 100$ Ω , $C_2 = 100$ nF $\rightarrow$ output low-pass

2.2.2 What to expect in simulation

Frequency domain (AC sweep):

- **Low frequencies:** Below $f_{HP} \approx \frac{1}{2\pi R_1 C_1} \approx 64$ Hz. Below this the gain falls (-20 dB/dec)
- **Midband:** Flat gain ~ 11
- **High frequencies:** Output RC cuts above $f_{LP} = \frac{1}{2\pi R_2 C_2} \approx 15.9$ kHz, plus the op-amp's own GBW will further roll off

Time domain:

- Apply a sine sweep or a step: you'll see low frequencies strongly attenuated, mid-band amplified by 11, high frequencies rolled off/smoothed
- For a step input: capacitor $C_1 + R_1$ cause slow baseline restoration (time constant ~ 2.5 ms)
- Output will show amplified AC portion; DC offset is removed

2.2.3 Waveforms to plot

- V_{in} (before C_1) and V_+ (after C_1) to see coupling effect
- V_{out} (op-amp output) steady state sine and step response
- For large amplitude fast waveforms, check slew limiting and distortion

2.2.4 Simulator gotchas

- Power rails matter $\rightarrow$ give the op-amp model the correct $\pm V$ rails
- Op-amp model selection $\rightarrow$ pick a realistic model with GBW and slew rate
- AC simulation $\rightarrow$ run .AC sweep from 1 Hz to 100 kHz to capture HP and LP corners
- Transient simulation $\rightarrow$ use time step small enough to resolve high-frequency content

2.3 Circuit 3: Counter + DAC + Comparator (Ramp/Count ADC)

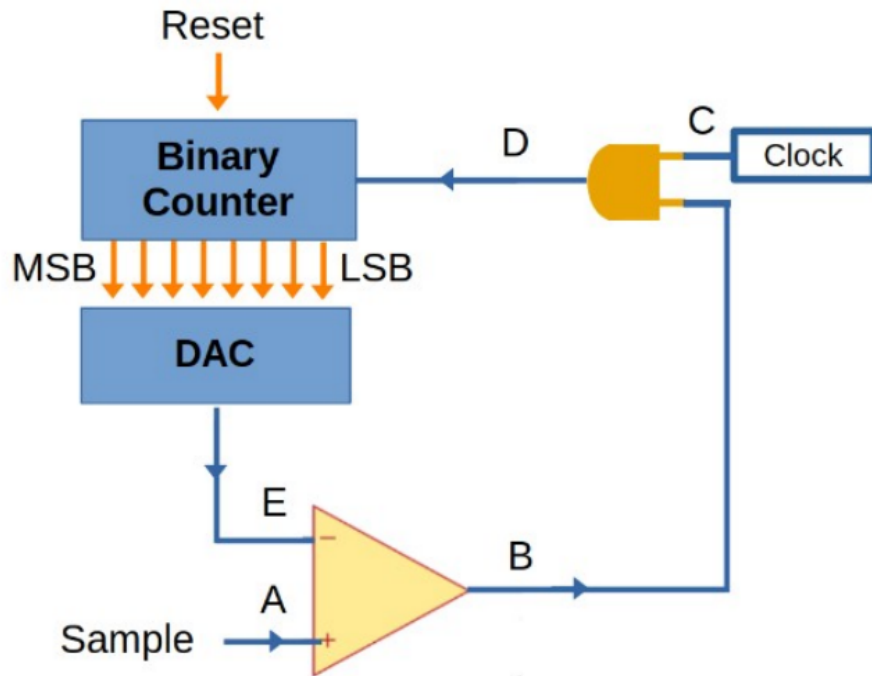


Figure 3: Circuit 3: Ramp ADC with Counter, DAC, and Comparator

2.3.1 What it is

A simple counting ADC: we hold the analog input, then count up a binary counter that drives a DAC. The DAC output rises in steps. A comparator watches DAC vs held sample; when $\text{DAC} \geq \text{sample}$, comparator stops the clock and the counter value is our digital result.

2.3.2 Expected simulation behavior (timeline)

1. **Reset and sample:** Put counter to 0, sample the input on an S/H capacitor. Comparator sees $\text{DAC}(0) < \text{sample}$.
2. **Enable clock:** Each clock tick increments counter $\rightarrow$ DAC steps up (staircase).
 - *Waveform:* DAC output is a staircase ramp; counter bits toggle in binary pattern.
3. **Comparator trip:** When DAC step crosses sample amplitude, comparator output flips and gate stops the clock.
 - *Waveform:* Comparator output shows a digital transition; DAC staircase stops at the crossing step.
4. **Read the counter:** That count is the measured code.

2.3.3 Key numbers & practical example

- Let DAC full scale = V_{ref} . For N-bit DAC, $LSB = V_{ref}/2^N$
- Example: $N = 8$, $V_{ref} = 5V \rightarrow LSB = 5/256 \approx 19.53 \text{ mV}$
- If sample = 1.0 V, expected code $\approx 1.0/0.01953 \approx 51$ (decimal)
- Conversion time worst case = 2^N clocks (if sample $\approx$ full scale)

2.3.4 Waveforms to probe

- Analog sample voltage (held): flat during conversion
- DAC analog output: staircase rising step by step
- Comparator input pins: crossing event visible
- Comparator output: digital high/low that gates the clock
- Counter outputs: binary waveform; you can display decimal too

2.3.5 Simulation pitfalls we expected

- S/H must hold during whole conversion
- DAC settling time $\rightarrow$ ensure clock period is long enough
- Comparator propagation delay and metastability
- Clock gating spikes $\rightarrow$ the gate that stops the clock must not produce extra pulses
- Timing resolution $\rightarrow$ set transient timestep small enough

3 The Tool Journey

Now that you know what we were trying to simulate, here's the story of how we tried (and tried, and tried) to actually do it.

3.1 Act 1: Proteus – The Obvious Choice (But Not an Option)

3.1.1 Why Proteus?

Proteus is one of the most common tools used for circuit simulation, especially in universities. It is:

- Easy to use
- Has a nice interface
- Supports both analog and digital circuits
- Perfect for our project

3.1.2 The problem?

Proteus is paid software.

Yeah, cracked versions are everywhere. We could've downloaded one in 5 minutes. But here's the thing:

From a cybersecurity point of view:

- Cracked software can contain malware or backdoors
- It breaks software license rules
- It goes against ethical and professional values

We're cybersecurity students. So we said no to cracked Proteus, even if it was the easiest solution.

Decision: Find a legal, free alternative.

3.2 Act 2: LTspice – Free, Powerful, and... Painful

3.2.1 Why LTspice?

LTspice is a free and very powerful circuit simulator. It's used by many engineers and companies, and it's known for accurate analog simulation.

Perfect, right?

3.2.2 First experience: Confusion

At the beginning, LTspice was *hard*:

- The interface is not very friendly
- Many things depend on shortcuts
- Error messages are confusing for beginners
- You must understand how SPICE works, not just draw circuits

3.2.3 Learning phase

To understand LTspice, we followed this tutorial playlist: <https://www.youtube.com/watch?v=16I71kfHNGU&t=281s>

We spent about one hour learning:

- How to draw circuits
- How to connect components correctly
- The importance of grounding
- How to run simulations and read results

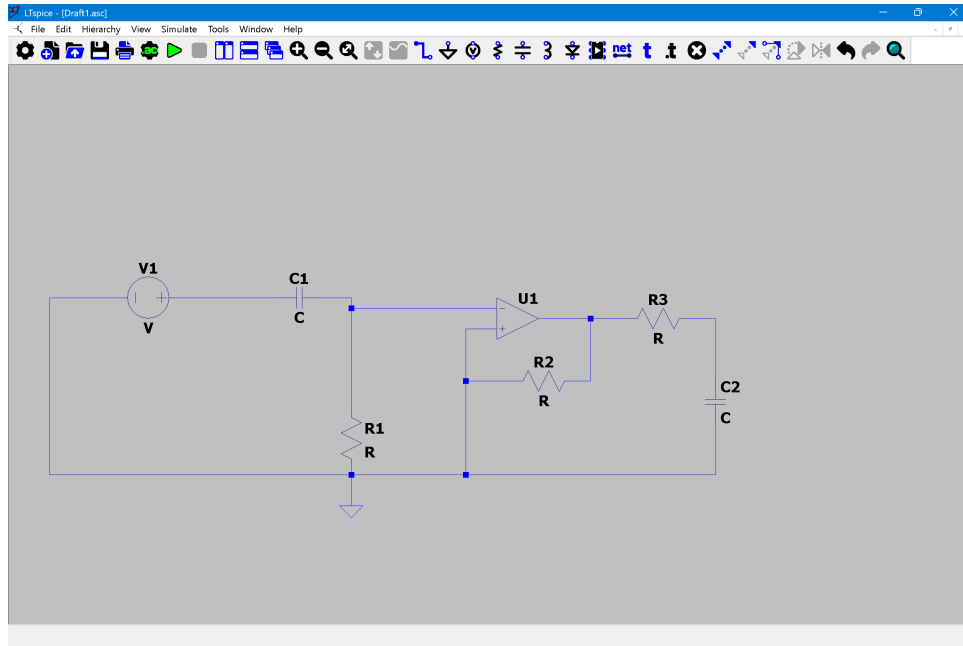


Figure 4: LTspice learning interface

This helped us understand the basics, but it was only the beginning.

3.2.4 Simulation problems: The nightmare

When we tried to simulate our *real* circuits, we faced many errors:

- Missing ground connections
- Floating nodes
- Op-amps not powered correctly
- Switches and digital parts needing special models
- Error messages that did not clearly explain the problem

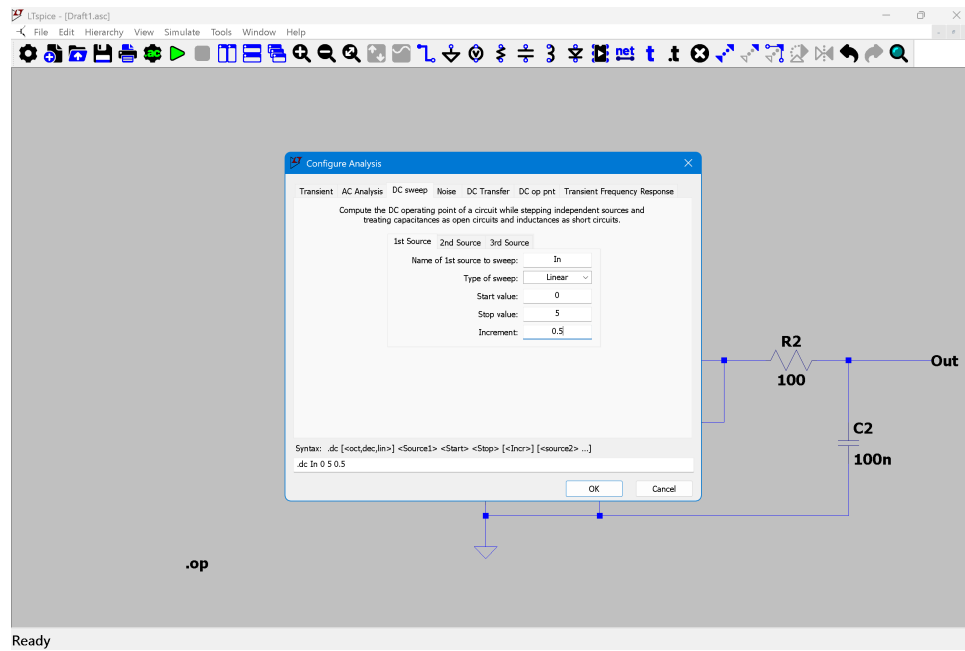


Figure 5: LTspice error messages

Instead of analyzing the circuit behavior, we spent most of the time debugging the simulator itself.

This made progress slow and frustrating.

Decision: We decided to move on to another tool.

3.3 Act 3: DesignSpark PCB – Great Design, Terrible Simulation

3.3.1 Why DesignSpark PCB?

After LTspice, we tried DesignSpark PCB, a free tool used for schematic design and PCB creation.

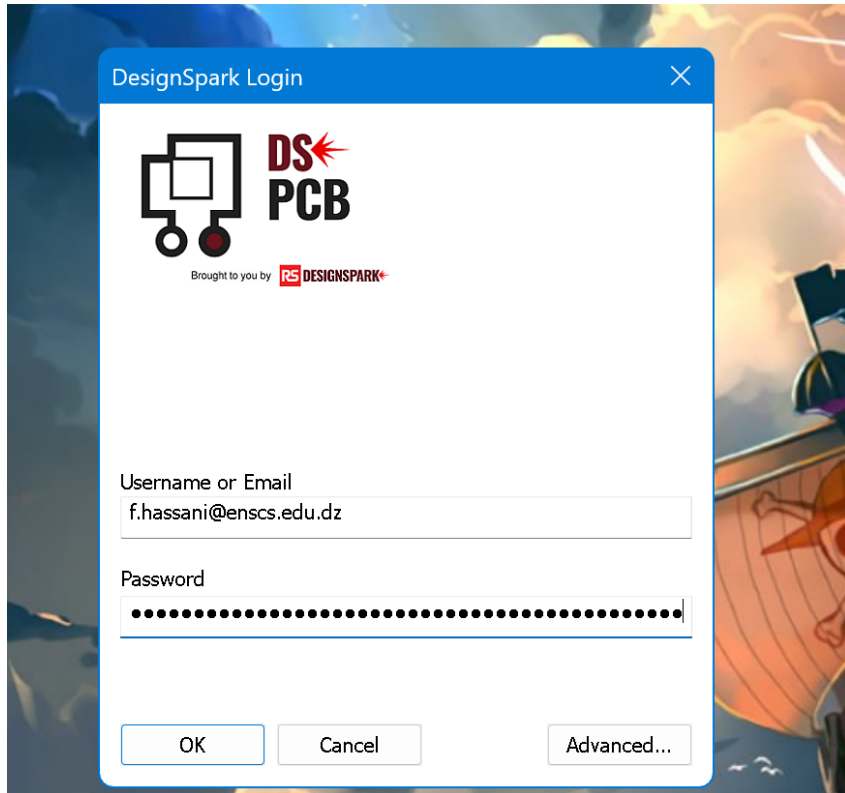


Figure 6: DesignSpark PCB interface

Why we chose it:

- Free and legal
- Easier to use than LTspice
- Clear schematic drawing
- More user-friendly interface
- Good for organizing and documenting circuits

3.3.2 Our goal:

- Redraw circuits clearly
- Better understand connections
- Prepare clean schematics for simulation or future work

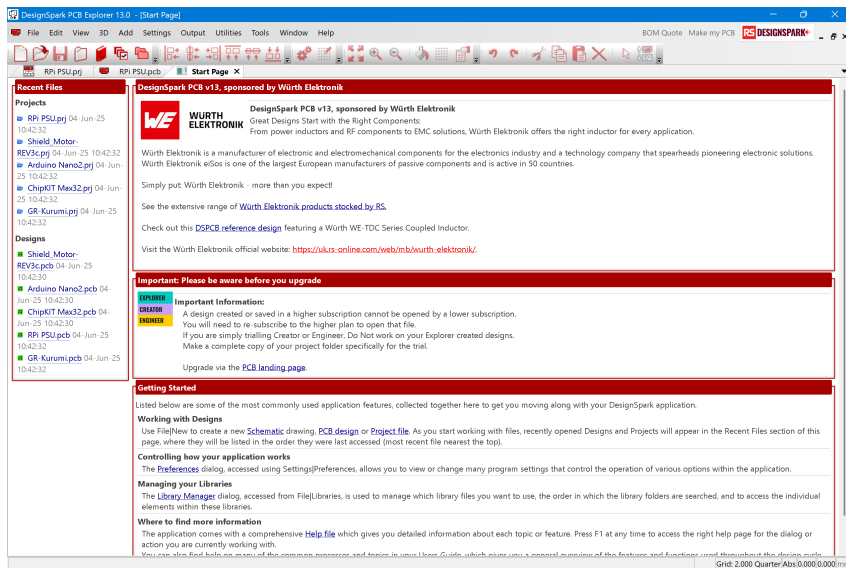


Figure 7: Circuit schematic in DesignSpark PCB

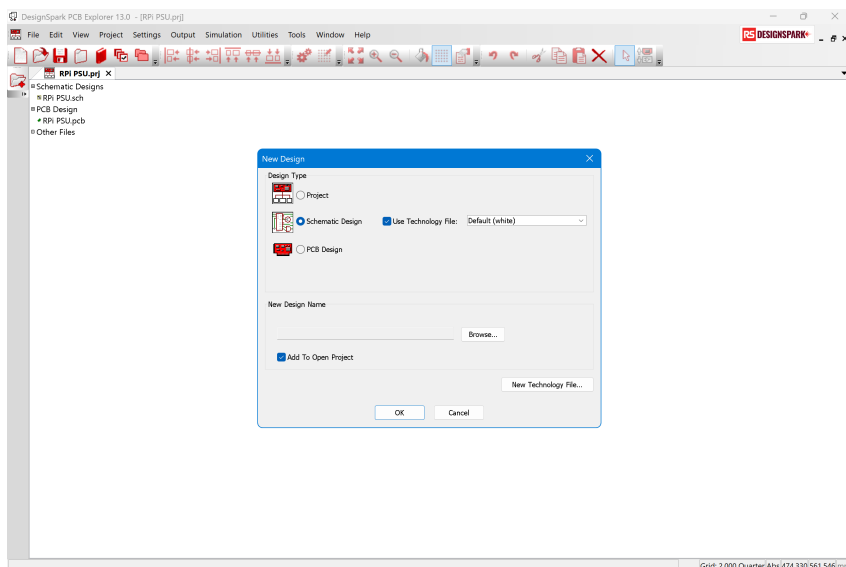


Figure 8: Another circuit view

3.3.3 Design experience: Great!

DesignSpark PCB was very easy and practical to use:

- Drawing circuits was fast and clean
- The interface was simple and well organized
- Connections were clear and easy to follow
- Schematics looked professional and readable

From a design perspective, DesignSpark PCB worked very well.

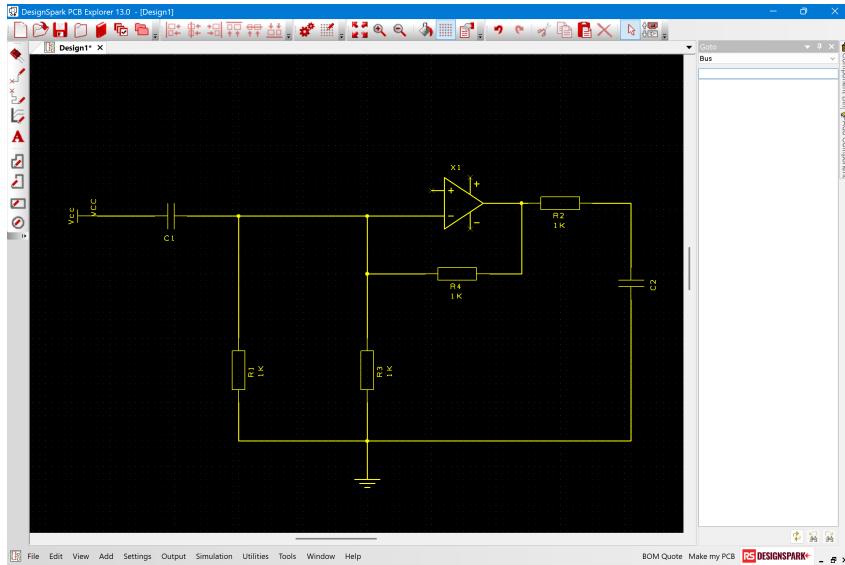


Figure 9: Professional schematic output

3.3.4 The simulation nightmare: Not great.

The real problem appeared when we tried to simulate the circuits.

Simulation in DesignSpark PCB is not straightforward:

- Simulation setup is complicated
- Documentation is limited
- Error messages are unclear
- Mixed analog and digital circuits are hard to simulate

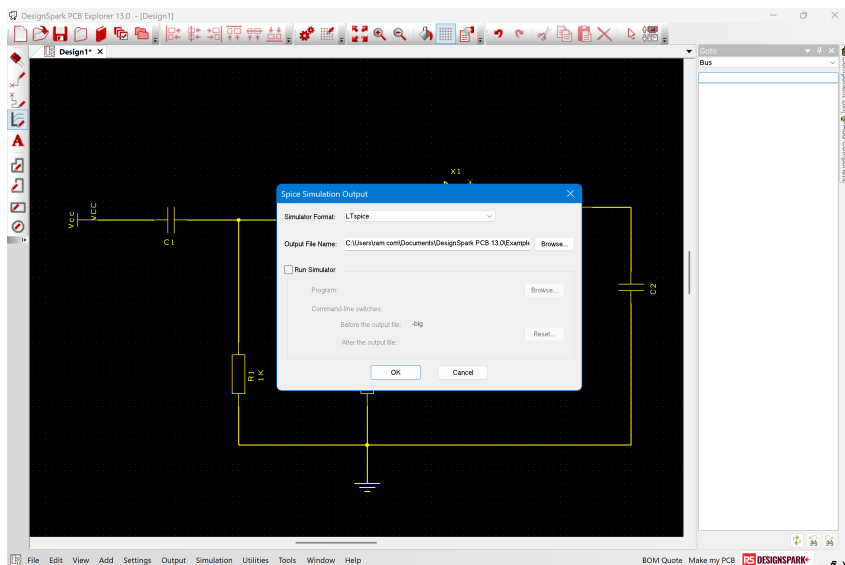


Figure 10: Simulation configuration challenges

We spent most of the night trying to make simulations work:

- Changing settings
- Testing small examples
- Searching online for solutions
- Rebuilding circuits multiple times

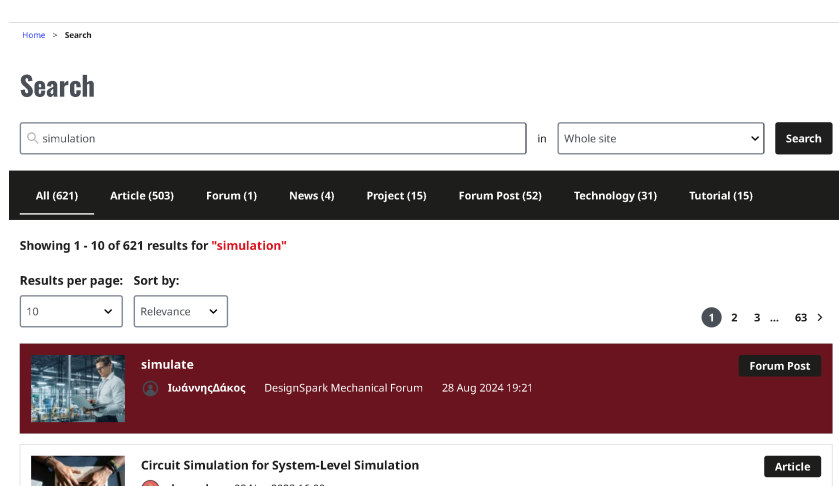


Figure 11: Late night debugging session

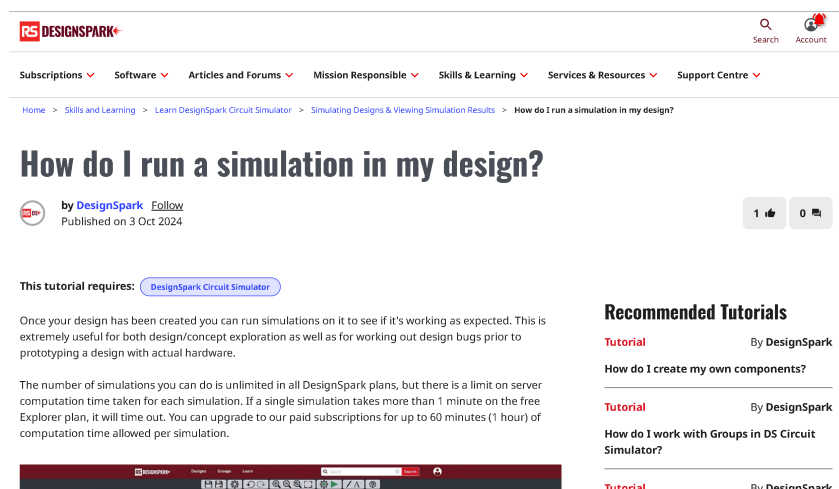


Figure 12: More simulation errors

Nothing worked as expected. Instead of analyzing voltages and signals, we were stuck dealing with software issues.

At that point, we were exhausted and out of ideas.

Decision: We made a very human engineering decision.

We stopped and went to sleep.

3.4 Act 4: Morning Clarity and the Proteus Demo

3.4.1 The next morning

With fresh minds, we looked at the problem differently. Instead of searching for a “perfect” tool, we searched for a *working and ethical solution*.

That’s when we found:

Proteus Free Trial

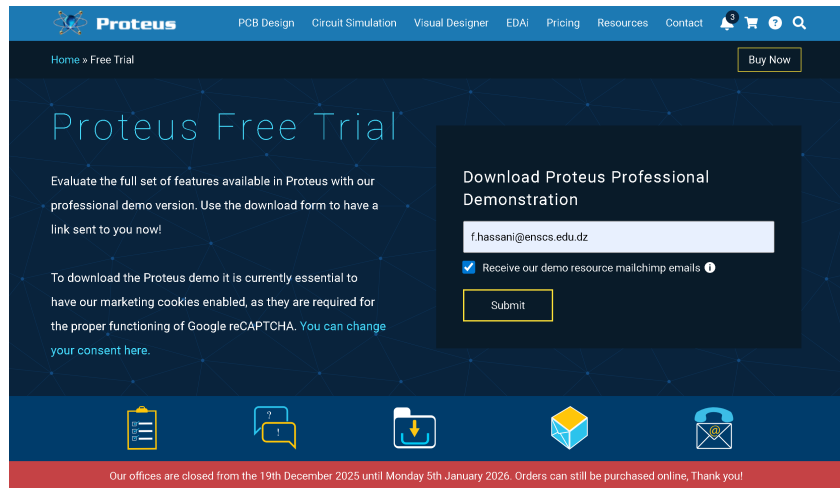


Figure 13: Proteus free trial discovery

Proteus provides a limited free trial that is:

- Legal
- Safe
- No cracked software involved
- Suitable for our simulation needs

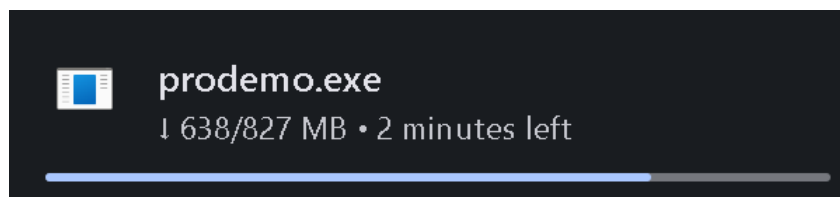


Figure 14: Trial version interface

This solution allowed us to continue the project without compromising our values or wasting more time.

3.5 Using Proteus Demo: Fast Success... and a New Limitation

3.5.1 First impression: Finally!

The difference was immediate.

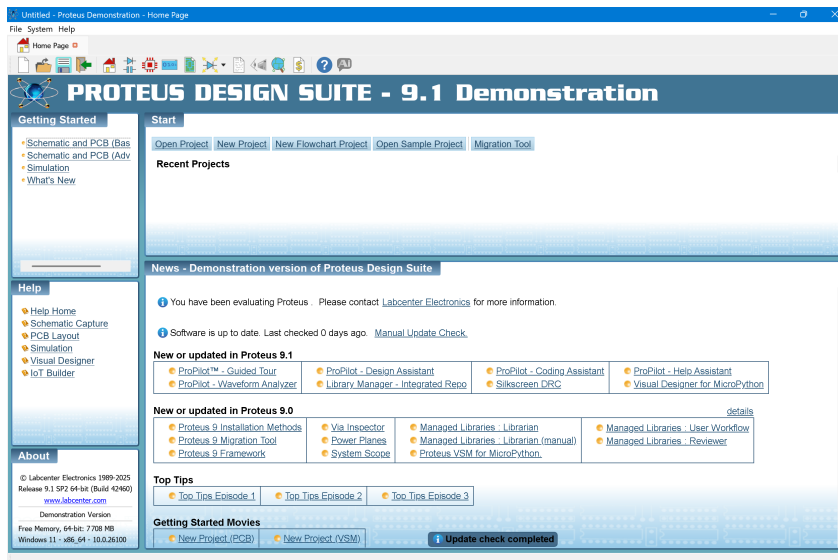


Figure 15: Working simulation in Proteus

Within 15 minutes, we were able to:

- Draw the first circuit
- Place all components correctly
- Set component values
- Run the simulation
- Observe waveforms and behavior

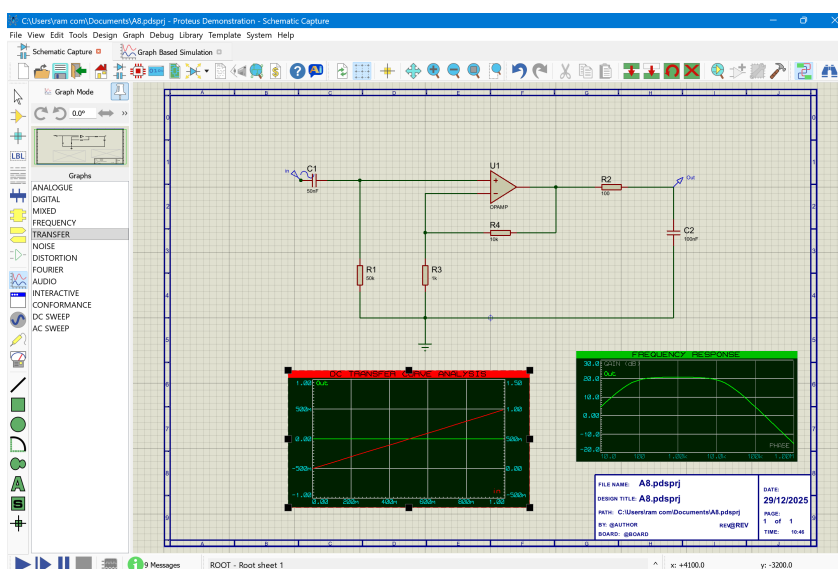


Figure 16: Successful waveform display

Compared to the *hours* spent struggling with other tools, Proteus felt fast, smooth, and intuitive.

For the first time, we were not fighting the software – we were actually analyzing the circuit.

At this point, everything looked perfect.

3.5.2 The “ProDemo” reality check

Then came the surprise.

When we tried to save our work, Proteus displayed a message from ProDemo mode:

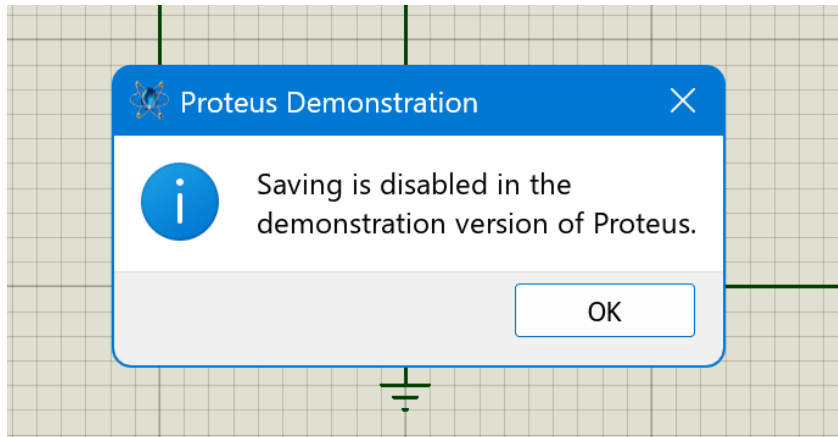


Figure 17: Save limitation message

Saving project files is not allowed in the demo version.

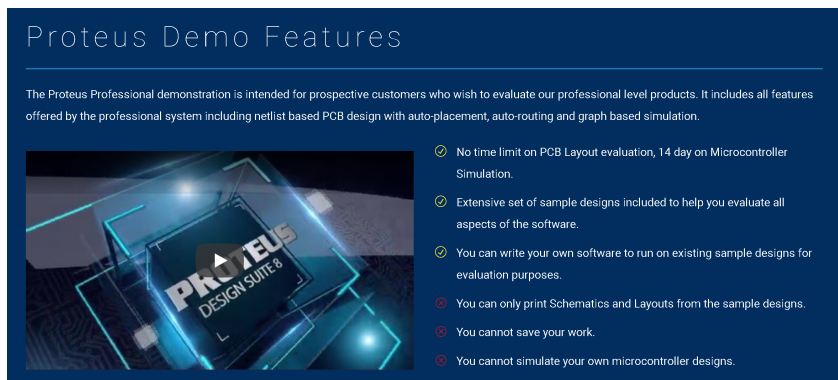


Figure 18: Demo version restrictions

This meant:

- We could simulate the circuit
- We could see results
- But we could not save the design files

In simple words: The simulation worked ... but nothing could be stored.

3.5.3 Impact on the project

This limitation created a new challenge:

- Any modification requires rebuilding the circuit again
- Progress cannot be stored normally

3.5.4 How we adapted

Instead of giving up again, we adapted:

- We focused on simulating one circuit at a time
- We took screenshots of schematics and waveforms
- We carefully documented results immediately
- We minimized closing the software during work sessions

This approach allowed us to continue working within the limits of the demo version, while still collecting valid results for the report (to submit the simulation files).

3.6 The Final (Final) Solution

At this point, we were tired. Tools were fighting us. Simulators were winning. Hope was... low.

So I (Fateh) said the forbidden words:

“Rock paper scissors... loser installs the cracked version.”

For about 2 seconds, this sounded like a genius idea.



Figure 19: Message by Fateh

Then reality hit. Cybersecurity brain activated.

Raid said nope!

3.6.1 Plot twist: Belkacem exists

Then we remembered something very important.

Belkacem already had Proteus installed on his computer. From before. No new downloads. No sketchy cracks at 3 a.m. No malware roulette.

So the plan became very simple:

- We use Belkacem’s computer
- We simulate everything there
- We save files normally

- We finish the project
- We keep our souls clean

As the legend says:

Belkacem was never in danger. Belkacem is the danger.

Instead of fighting software licenses, we went back to: circuits, signals, and logic (the good stuff).

4 What We Learned: Hard Skills Edition

Even through the chaos, we learned real technical skills.

4.1 Circuit Simulation Basics

- How simulators work
- Why grounding is mandatory
- How missing references cause errors
- The difference between ideal theory and simulation reality

4.2 SPICE Fundamentals

- Basic understanding of nodes and netlists
- What floating nodes are and why they break everything
- Convergence errors and how to fix them
- Importance of simulation models for components

4.3 Analog Circuit Behavior

- How op-amps, capacitors, resistors, and switches behave differently in simulation vs theory
- Power rail requirements
- Slew rate and bandwidth limitations
- Charge injection and clock feedthrough effects

4.4 Mixed-Signal Simulation Concepts

- Difficulty of simulating analog and digital blocks together
- Importance of proper timing
- Clock period vs settling time
- Comparator delays and metastability

4.5 Debugging Simulation Errors

- How to identify whether a problem comes from circuit design or simulation setup
- Reading cryptic error messages
- Adding series resistances and GND references to fix convergence
- Checking power supplies and initial conditions

4.6 Tool Comparison and Selection

Evaluating tools not only by features, but by:

- Usability
- Learning curve
- Project constraints
- Legal/ethical considerations

4.7 Reading Documentation and Technical Tutorials

- Learning new tools using documentation, videos, and online resources
- Not relying only on intuition
- Following structured learning paths

4.8 Fast Schematic Building

- Quickly recreating circuits from memory and diagrams
- Working efficiently when save functionality is unavailable

4.9 Efficient Simulation Workflow

- Running simulations and analyzing results quickly
- Documenting results in real time
- Capturing waveforms and screenshots immediately

4.10 Understanding Software Limitations

- How demo versions restrict features
- Working around limitations legally
- Judging risk like a cybersecurity professional

4.11 Teamwork and Adaptation

- Making decisions as a team
- Not doing dumb solo decisions at 3 a.m.
- Using available resources (like Belkacem’s computer)
- Adapting to unexpected problems

5 Quick Debugging Cheats for the sim

These are practical tips we learned the hard way:

5.1 “No convergence” or weird errors:

- Add small series resistance ($1\mu\text{--}1\text{k}\Omega$) on floating nodes
- Add GMIN steps in SPICE settings
- Check that all nodes have a DC path to ground

5.2 “Output oscillates”:

- Add output resistor (like R_2)
- Add small compensation capacitor across feedback
- Check power rail decoupling

5.3 “Comparator keeps toggling”:

- Add tiny hysteresis
- Slow the input ramp so DAC has time to settle
- Model comparator propagation delay

5.4 “Counter miscounts”:

- Lengthen clock period so DAC and comparator settle before counting next tick
- Check edge-triggered vs level-triggered logic
- Add debouncing on comparator output

5.5 “Switch doesn’t work”:

- Use proper transmission gate models with complementary control signals
- Ensure clock signals have proper rise/fall times
- Check switch on-resistance and off-capacitance

5.6 “Op-amp acts weird”:

- Check power supply rails are connected
- Use realistic op-amp model (not ideal)
- Verify input bias resistor network
- Check frequency compensation

6 Key Lessons from This Experience

This step taught us that engineering is not only about knowing formulas:

- **Tools matter:** The right tool can save hours of work
- **Time matters:** Learning curve must fit project deadlines
- **Ethics matter:** Taking shortcuts can cost you more later
- **Learning how to adapt matters** – A working solution beats a perfect one that never works

6.1 The struggle itself improved our technical skills

Instead of just reading theory, we:

- Debugged real simulation problems
- Learned multiple tools under pressure
- Made engineering decisions with real constraints
- Worked as a team through frustration

This is what real engineering looks like.

7 Final Wisdom from This Adventure

This project taught us one big truth:

Engineering isn’t about having the perfect tools or conditions. It’s about:

- Problem-solving under constraints
- Making ethical decisions when shortcuts are tempting
- Learning from failures faster than giving up
- Working together to find creative solutions

We didn’t just simulate three circuits. We learned how to be better engineers.

8 Conclusion

We started this project thinking it would be about:

- Op-amps
- Capacitors
- DACs
- Waveforms

Instead, it became a lesson about:

- Tool selection
- Ethical engineering
- Teamwork
- Perseverance

Did we simulate the circuits? Yes.

Did we learn more than expected? Absolutely.

Was Belkacem the hero we needed? 100%.

The simulation files are attached with this report. They work. They're legal. And they taught us more than just circuit theory.

Submitted by:

Hassani Fateh & Kahlrras Raid

To:

Mr Sahraoui Walid & Mr BOUCHACI Islem

With special thanks to:

Belkacem (the legend)